Separating task information from evaluation effects in language-model performance*

Majekodunmi Ajiboye

Abstract

A model's score depends on both the problem it is given and the conditions under which its answer is accepted. We study these two sources separately. Study 1 is response-conditioned: for each of 16,108 ADeLe items, a predictor already sees the other fourteen models' outcomes and then receives eighteen published task characteristics. In the descriptive structured-feature analysis, the added task information reduces log loss by 1.74% under task-family holdout and 1.68% when whole benchmarks are withheld from fitting. The benchmark-held-out gain is positive on sixteen of twenty benchmarks and remains 1.68% with nested regularisation. Generic text representations are less stable. The prespecified primary full-embedding comparison worsens prediction in the frozen analysis and remains worse after a post hoc grid extension to stronger penalties. Study 2 uses 72 reserved programming tasks. Adding an output scaffold raises Gemma's requested-wrapper compliance from 50% to 100%, while wrapper-tolerant test success changes only from 38.9% to 40.3%. For Gemma's SPEC-versus-NL comparison, strict scoring yields a 25.0-point advantage, whereas tolerant scoring of the same generations yields 2.8 points. Sixteen NL responses rejected by strict extraction pass the packaged tests after mechanical wrapping, with the generated function text unchanged. Neither model's primary semantic-success contrasts meet the prespecified threshold. The results support a practical separation: ask what task information adds to prediction, then ask how presentation and acceptance rules change the score before interpreting that score as problem-solving capability.

1 Introduction

A language model can fail a programming benchmark because its algorithm is wrong. It can also fail because a function that passes the packaged tests was returned outside the wrapper expected by the evaluator. The recorded outcome may be the same, but the explanation and the appropriate remedy are different. A similar ambiguity arises when predicting performance. A benchmark average describes success on a particular collection of items, yet it does not specify which properties of an individual item make that success likely. Understanding performance therefore requires more than recording a score. It requires distinguishing information about the task from the conditions that turn an answer into a scored outcome.

This distinction does not make aggregate evaluation uninformative. Low-dimensional response structure predicts substantial variation in language-model performance (Ruan et al., 2024). Methods based on that structure can recover skill profiles (Maimon et al., 2026) and reduce the number of examples needed to estimate benchmark scores (Maia Polo et al., 2024). The question is what remains to be learned once such regularities are already available. Task descriptions may explain variation that peer-model outcomes do not capture. Presentation and acceptance rules may also affect scores in ways that should not be attributed to the underlying solution alone.

We investigate these possibilities through two complementary studies. The first supplies a performance predictor with a strong response baseline: fourteen peer models' binary outcomes on the very item whose outcome is to be predicted for a fifteenth model. We then ask whether an existing set of structured task

*A large language model assisted with literature discovery, methodological critique, software development, statistical checking, and manuscript drafting. The author is responsible for the analyses, interpretations, and final text.

characteristics adds useful information. The second holds programming tasks and checkpoints fixed while changing the supplied input, and distinguishes requested-format compliance from success under an evaluator that accepts a mechanically wrapped bare function. The studies address different quantities. One concerns prediction of observed outcomes. The other concerns the measurement process that produces them.

The first study builds directly on ADeLe, which introduced general task-demand scales and demonstrated demand-based prediction, including transfer to new tasks and benchmarks (Zhou et al., 2026). We extend that comparison to a response-conditioned setting. Structured annotations and generic text representations are added to the same full peer-response baseline and evaluated with a common downstream estimator. Whole-benchmark holdout asks whether their incremental value survives removal of the target benchmark from fitting.

The second study uses the programming materials released with VeriContest (Xie et al., 2026). Prompt sensitivity is well established (Sclar et al., 2024). More directly, prior work has separated task performance from output-format adherence across multiple formats, including wrappers (Long et al., 2025), and has shown that output extraction can materially change execution-based code scores (Macedo et al., 2024). Our question is narrower. We ask how much of an observed prompt-condition gap can be assigned exactly to a fixed acceptance rule when the saved generation is held constant. A prospective reserved-task experiment then tests scaffold exposure separately from the uncertain benefit of adding specification content.

The results give the distinction practical content. Structured task characteristics retain a modest conditional predictive gain on held-out benchmarks. A substantial programming score difference can be accounted for by a known acceptance rule applied to the same outputs. One result identifies information that remains useful under a harder predictive test. The other identifies why an apparent improvement changes when the scoring rule changes. Together they make model performance more interpretable by separating the task information used to predict an outcome from the conditions used to elicit and measure it.

2 Two questions about the same score

The task-information question is deliberately conditional. We do not ask whether task characteristics predict performance by themselves. Instead, the predictor first receives a rich response profile for the item, and we ask whether structured task information adds anything beyond it. This matters because a standalone task model can succeed simply by encoding general difficulty. Let Y_{mi} be the scored outcome of model m on item i, $R_{i,-m}$ the available peer-response information, and D_i the structured task characteristics. We compare held-out losses from predictors using R and (R, D):

$$\Delta_D = \widehat{\mathcal{L}}(R) - \widehat{\mathcal{L}}(R, D). \tag{1}$$

A positive value means that the additional features improve prediction in the tested pipeline. It does not identify the causal effect of changing a task characteristic. Nor is it an estimate of how much better the language model itself performs when given those features. The annotations are supplied to the external predictor, not to the model solving the task.

The evaluation-effects question instead concerns changes to the model's input and to the rule applied to its output. We distinguish three observable endpoints in programming: whether the requested wrapper is emitted, whether the accepted implementation compiles, and whether it passes every packaged positive test. These endpoints are related, but they are not interchangeable. A prompt can improve the first without establishing an improvement in the third.

This separation also determines the controls. Study 1 keeps the response information constant across feature comparisons and excludes target benchmark outcomes from fitting. Study 2 compares input conditions on the same tasks and applies strict and tolerant acceptance rules to the same generated responses. The first design tests incremental prediction under distribution shift. The second permits an exact accounting of the score change caused by the specified acceptance rule. Neither is a general causal model of all sources of model performance.

3 Task characteristics retain value beyond peer responses

3.1 A demanding conditional baseline

We use the released ADeLe response matrix and canonical annotations (Zhou et al., 2026). The analysis retains items with finite binary outcomes for all fifteen models, complete task annotations, and unguessability UG ≥ 75. The resulting set contains 16,108 items from 63 task families and twenty benchmarks. Taking each model as target gives 241,620 model–item predictions. These are correlated observations from a fixed model panel, not that many independent experimental units.

For each target model, the baseline receives the other fourteen models' outcomes on the item, a Jeffreys-smoothed logit of their mean success, and benchmark identity. It is fitted separately for each target. The target model's own outcome never enters its peer features. The baseline therefore has much richer information than a single aggregate difficulty score, including which peers succeeded rather than only how many.

The prediction setting is deliberately response-conditioned. Peer outcomes on held-out items remain available at prediction time. This is useful for completing or interpreting an existing response matrix. It is not cold-start prediction before any model has attempted an item. Holding a benchmark out excludes its target outcomes from fitting, not all observations about its test items. This information boundary is part of the question rather than a technical exception to it.

3.2 Task representations and held-out evaluation

The structured representation D consists of eighteen published ADeLe coordinates. They cover rubric-defined demands and task characteristics, including volume and atypicality. We treat them as an inherited representation, not as independently verified measurements of eighteen causal mechanisms. Unguessability is used for filtering only.

The generic alternatives use a frozen Nomic text encoder (Nussbaum et al., 2025). We embed the canonical question field with the prefix `classification:`. The full representation, T_{768}, retains all 768 embedding coordinates. The compact representation, T_{18}, retains eighteen principal components fitted only to training items in each outer fold. This controls coordinate count in the compact comparison, although it does not equate the information, cost, or effective complexity of the two representations. We evaluate R, $R + D$, $R + T_{18}$, $R + T_{18} + D$, $R + T_{768}$, and $R + T_{768} + D$.

All predictors use L2-regularised logistic regression. Numerical scaling and benchmark encoding are fitted only on training data. The fixed analysis uses $C = 1$, the `liblinear` solver, and a maximum of 4,000 iterations. We evaluate both leave-one-task-family-out prediction across 63 folds and leave-one-benchmark-out prediction across twenty folds. In the latter, the unseen benchmark maps to an all-zero benchmark-indicator block. No held-out benchmark row is used to fit the predictor or the PCA.

A nested sensitivity selects C from $\{0.01, 0.1, 1, 10\}$ separately for each target, representation, and outer benchmark fold. Selection uses three inner folds grouped by benchmark. Scaling, encoding, and PCA are refitted inside each inner training split. All six representations use the same inner splits and selection criterion.

The transfer extension's prespecified primary comparison was R against $R + T_{768}$. The D comparisons were designated descriptive analyses of an existing representation. We report that distinction because a useful descriptive result does not change which primary comparison was specified. The regularisation grid was fixed before the extension ran. The exact inner-fold implementation was settled after the fixed-C results were available, but before the sensitivity ran. Supplementary Section S1 records the analysis status of every component.

3.3 A modest gain that survives benchmark holdout

The structured characteristics improve prediction in both evaluation settings (Table 1). Under task-family holdout, log loss decreases from 0.35665 to 0.35045, a relative reduction of 1.74%. Under whole-benchmark

holdout it decreases from 0.37402 to 0.36775, a reduction of 1.68%. The corresponding absolute gains, 0.00620 and 0.00628, are similar despite the harder split.

Table 1: Prediction from peer responses with and without task characteristics. Lower log loss is better. The task-family and fixed-C benchmark columns use the same prefixed embeddings and estimation procedure. The final column selects regularisation inside each training fold. All six evaluated representations are shown.

Predictor	Task-family holdout Fixed $C = 1$	Benchmark holdout Fixed $C = 1$	Benchmark holdout Nested C
R	0.35665	0.37402	0.37253
$R + D$	0.35045	0.36775	0.36627
$R + T_{18}$	0.35144	0.37203	0.36831
$R + T_{18} + D$	0.34985	0.36963	0.36660
$R + T_{768}$	0.39325	0.45564	0.38647
$R + T_{768} + D$	0.39271	0.45751	0.38717

The fixed-C benchmark-held-out demand gain is positive on sixteen of twenty benchmarks. Its equal-benchmark mean is 0.00723, compared with the item-weighted mean of 0.00628. The benchmark-cluster resampling interval is [0.00204, 0.01100]. Twelve of the fifteen target models improve. The result is therefore neither universal nor confined to a single target model. The harder split also reduces concentration. MMLU-Pro accounts for 66.8% of the summed demand gain under task-family holdout and 30.7% under benchmark holdout. Supplementary Section S3 gives every benchmark-specific contrast.

Nested regularisation leaves the main pattern intact. The baseline loss is 0.37253 and the demand-augmented loss is 0.36627. Their difference is 0.00625, again a relative reduction of 1.68%, with a benchmark-cluster interval of [0.00126, 0.01122]. Sixteen benchmarks and twelve targets improve. The Brier score also decreases, from 0.11728 to 0.11553. This agreement across proper scoring rules supports an improvement in probabilistic prediction rather than an effect limited to a chosen classification threshold (Gneiting & Raftery, 2007).

These analyses do not constitute independent replications. They reuse the same items, labels, and model panel under different fitting and evaluation procedures. Their value is that a specific alternative explanation becomes less adequate: the observed increment cannot depend only on fitting a coefficient for a benchmark that is already present in training. The result still permits shared content, formatting, or other regularities across benchmarks. It establishes conditional predictive transfer within this battery, not causal isolation of task demands.

3.4 More text features do not automatically improve prediction

The full generic embedding does not pass the primary transfer comparison. With fixed regularisation, its benchmark-held-out loss is 0.45564, against 0.37402 for R, and it is worse on all twenty benchmarks. It also underperforms in the task-family setting, where its loss is 0.39325. The failure therefore cannot be attributed solely to excluding whole benchmarks.

Nested selection reduces the deficit but does not remove it within the original grid. Full-text loss is 0.38647 against 0.37253 for its tuned baseline, a 3.74% increase, with interval $[-0.02175, -0.00131]$ for $L(R) - L(R + T_{768})$. All 300 fits select the lower boundary $C = 0.01$, leaving open whether stronger regularisation would alter the result.

We therefore ran a post hoc sensitivity adding $C = 0.001$ and $C = 0.0001$ with the same folds and selection procedure. The new lower boundary is never selected. For $R + T_{768}$, 196 fits choose $C = 0.01$ and 104 choose $C = 0.001$. Full-text loss remains worse at 0.38687 against 0.37253 for R, a 3.85% increase. Seventeen of twenty benchmarks are worse and the interval is $[-0.02136, -0.00197]$. The extended grid therefore does not explain the deficit. This sensitivity is post hoc and does not change the frozen primary status. Other encoders and estimators remain open.

The compact text representation is less decisive. Its benchmark-held-out relative gain is 0.53% with fixed C and 1.13% after nested selection, with both intervals including zero. Under task-family holdout it is positive

on Date Arithmetic, ChemLLMBench, and Math, the three benchmarks where D is negative. The pattern is representation dependent but does not identify why. Directly comparing D with compact text favours D by 0.00428 at fixed C, with interval [0.00113, 0.00624], and by 0.00204 after tuning, with interval [−0.00102, 0.00376]. These post-result audits are reported in Supplementary Section S4.

The appropriate conclusion is that the structured representation has a stable positive increment under the tested controls. Its superiority to compact text is not established consistently across regularisation procedures. Combining the two gives no clear improvement over D alone under benchmark holdout. The evidence favours retaining task characteristics as predictive inputs, while leaving the best representation and estimator open.

3.5 What the uncertainty estimates describe

We assess loss differences on paired saved out-of-fold predictions. Each resampling draw selects twenty whole benchmark clusters with replacement and recomputes the item-weighted difference. The reported bounds are the 2.5th and 97.5th percentiles of 10,000 draws. Target models, compared predictions, and outcomes remain together within each selected cluster.

These intervals describe sensitivity to benchmark composition conditional on the fitted predictions and observed model panel. They do not refit the complete pipeline, and they do not provide unconditional coverage for arbitrary new benchmarks or checkpoints. Overlapping training folds introduce dependence that ordinary cross-validation uncertainty calculations can miss (Bates et al., 2024). The sign counts, equal-benchmark means, and complete contrast tables supplement rather than eliminate that limitation. The 1.68% figure is a reduction in prediction loss, not a 1.68-percentage-point improvement in the language model's task success.

4 Input scaffolds change what programming scores record

4.1 Separating the supplied content from the acceptance rule

A specification-containing prompt can supply several things at once: formal constraints, helper definitions, a longer description, and an example of the required output structure. An observed score difference between specification and natural language does not identify which of these components mattered. We use a four-condition experiment to separate scaffold exposure from the addition of the complete specification, and a paired scoring comparison to isolate the consequence of wrapper acceptance.

The tasks come from VeriContest's LeetCode and Codeforces materials (Xie et al., 2026). A deterministic, outcome-blind split reserved 72 tasks after a 24-task development pilot. The reserve contains eighteen Codeforces and 54 LeetCode tasks. Both retained local checkpoints, Devstral Small 2 24B and Gemma 3 12B, attempt every reserved task under four inputs. NL contains the natural-language description. NL+scaffold adds the requested `Solution` declaration and target function structure, without formal constraints or specification helper functions. SPEC supplies the original formal specification. BOTH combines the natural-language description and specification.

The requested output is a Rust implementation in an `impl Solution` block, without proofs or Verus annotations. The common instruction, target signature, generation limits, and testing procedure are fixed across arms. Each cell contains one generation under temperature-zero decoding, a fixed seed, and a 4,096-token output limit. There is no compiler feedback or repair round. The resulting 576 cells are complete. Exact model-build identifiers and execution settings appear in Supplementary Section S5.

The primary evaluator first applies the original extractor. When a response lacks the wrapper but contains a bare public function, it encloses the unchanged generated text in `impl Solution`. Other extraction failures remain failures. This rule was first applied outcome-aware during the development-pilot audit, then fixed before generating any reserved-task responses. An accepted implementation succeeds only if it compiles and passes every packaged positive test. We call this *test success*. It is the operational semantic endpoint, not a proof of program correctness.

The two primary contrasts within each model are NL+scaffold minus NL and BOTH minus NL+scaffold. The first tests scaffold exposure. The second tests adding the whole specification while retaining the natural-language problem and an exposed wrapper. We use the exact conditional (binomial) McNemar test (McNemar, 1947; Fagerland et al., 2013). Each model's two primary tests are evaluated at the Bonferroni-adjusted threshold $\alpha = 0.025$. This controls the two-test family within a model, not a pooled family of four tests across both models.

4.2 Compliance changes much more than test success

Gemma emits the requested wrapper on 36 of 72 NL tasks and on all 72 NL+scaffold tasks. Test success under tolerant evaluation changes from 28 to 29. The input scaffold therefore raises observed wrapper compliance by 50.0 percentage points, while the observed test-success change is 1.4 points (Table 2). Compilation lies between these endpoints, increasing from 55 to 61 tasks, or 8.3 points.

Table 2: Reserved-task programming results. Each row contains 72 generations. Wrapper counts refer to unmodified output compliance. Compilation and test success use the tolerant evaluator. Strict success counts a missing wrapper as failure. The strict and tolerant scores are evaluated on the same responses.

Model	Input	Wrapper	Compiles	Test success	Strict success
Devstral	NL	72	64	39	39
	NL+scaffold	72	66	41	41
	SPEC	71	60	34	34
	BOTH	72	63	46	46
Gemma	NL	36	55	28	12
	NL+scaffold	72	61	29	29
	SPEC	72	62	30	30
	BOTH	72	65	36	36

The paired outcomes make the small net semantic change explicit. Gemma gains three test successes and loses two under NL+scaffold, giving an exact $p = 1.000$. Devstral gains seven and loses five, giving a 2.8-point change and $p = 0.774$. The additional-specification effects are positive in both models, but neither meets its primary threshold (Table 3). Gemma gains nine tasks and loses two, a 9.7-point difference with $p = 0.065$. Devstral gains fourteen and loses nine, a 6.9-point difference with $p = 0.405$.

Table 3: All prespecified primary test-success contrasts on the reserve. Gains are control failures that become treatment successes. Losses are the reverse. None meets the within-model threshold $\alpha = 0.025$.

Model	Contrast	Difference (pp)	Gains/losses	Exact p
Devstral	NL+scaffold minus NL	+2.8	7/5	0.774
Devstral	BOTH minus NL+scaffold	+6.9	14/9	0.405
Gemma	NL+scaffold minus NL	+1.4	3/2	1.000
Gemma	BOTH minus NL+scaffold	+9.7	9/2	0.065

These results distinguish a large observed compliance effect from an unestablished general semantic benefit. They do not establish that the semantic effects are exactly zero. In particular, the positive additional-specification estimates remain compatible with benefits that this 72-task comparison does not confirm at its chosen threshold. The compliance, compilation, and test-success differences describe separate endpoints. Their decreasing magnitudes are not a formal mediation analysis.

4.3 An exact accounting of the evaluator-dependent score gap

The same saved outputs permit a stronger statement about acceptance rules. Under strict extraction, Gemma succeeds on twelve NL tasks and thirty SPEC tasks. The specification advantage is therefore 18/72, or 25.0 percentage points. Under tolerant extraction, NL success rises to 28 while SPEC success remains thirty. The

difference is 2/72, or 2.8 points. Sixteen otherwise rejected NL responses pass after mechanical wrapping, without alteration of the generated function text.

This comparison has a simple exact decomposition. Let $s_i(e)$ and $t_i(e)$ be strict and tolerant success for item i under input e, and let $r_i(e) = t_i(e) - s_i(e)$ indicate a success recovered by the tolerant rule. For any two inputs a and b,

$$\underbrace{\overline{s(b) - s(a)}}_{\text{strict score difference}} = \underbrace{\overline{t(b) - t(a)}}_{\text{tolerant score difference}} + \underbrace{\overline{r(a)} - \overline{r(b)}}_{\text{differential acceptance penalty}} . \tag{2}$$

For Gemma's SPEC-minus-NL comparison, this is

$$25.0 \text{ pp} = 2.8 \text{ pp} + 22.2 \text{ pp}. \tag{3}$$

Thus 88.9% of the strict score gap disappears under the specified rescoring rule. This percentage is an accounting of the observed gap, not an estimate of a natural indirect effect or a population-wide mediation fraction. It requires no regression adjustment because the responses being scored are identical.

The difference is not a claim that strict extraction is always inappropriate. If an application requires the exact interface and offers no adaptation, failure to provide that interface is a real system failure. If the intended construct is correctness of the generated implementation after routine wrapper adaptation, strict extraction combines that construct with another requirement. Reporting both scores exposes the distinction. A single success number conceals it.

4.4 What remains uncertain about specification content

The experiment does not isolate formal notation from the rest of the specification. BOTH minus NL+scaffold adds constraints, helper definitions, and text length together. Context length can itself affect model performance (Du et al., 2025). No adjustment to the existing outcomes can identify separate causal effects for components that were not independently varied.

The reserve also limits the interpretation of the earlier development result. Devstral's SPEC-minus-NL effect was +37.5 points on the 24 development tasks but is −6.9 points on the reserve. Shared-arm templates, recorded checkpoint identity, and Ollama version match across phases. The large development advantage therefore did not transfer to the reserved tasks. Both phases have the same broad Codeforces/LeetCode proportions, so those proportions alone cannot account for the discrepancy. They do not rule out differences between the individual tasks.

Supplementary Section S7 reports the development audit and exploratory task-source breakdowns. They describe variation in the remaining semantic effects without replacing the primary conclusions. The acceptance result is established directly by rescoring the same outputs and by the prospective scaffold comparison.

5 Relation to existing evaluation work

The two studies connect task-based prediction with measurement-aware evaluation. ADeLe already supplies an interpretable task representation and demand-based assessors that generalise across tasks and benchmarks (Zhou et al., 2026). Our analysis asks whether the published representation remains useful after peer outcomes on each item are known. It therefore evaluates complementary information rather than reproducing ADeLe's original comparison against text-only assessors. The generic representations here use a common logistic estimator, unlike the original annotation random forest and fine-tuned text alternatives.

Response-based approaches address the complementary source of predictive information. Observational scaling laws exploit low-dimensional capability structure across models (Ruan et al., 2024). Low-rank skill analysis uses model-by-benchmark outcomes to profile strengths and redundancies (Maimon et al., 2026). Small representative test subsets can estimate much larger benchmark scores (Maia Polo et al., 2024). Our full peer-response baseline takes that redundancy seriously. A residual task-feature gain does not refute a strong common performance factor. It shows that the finite response-based predictors tested here do not exhaust the useful item information.

Prompt sensitivity is likewise not new. Meaning-preserving formatting choices can change model performance (Sclar et al., 2024), and multi-prompt evaluation can change conclusions about both absolute and relative performance (Mizrahi et al., 2024). Long et al. make the closer distinction between performance when output-format constraints are obeyed and performance scored regardless of format adherence, across fifteen formats that include wrappers (Long et al., 2025). Macedo et al. show a code-specific version: mixed code and surrounding text can depress execution-based code-translation scores unless the source code is reliably extracted (Macedo et al., 2024). The expanded study also examines text-based metrics and output-format variation in closed-source models (Macedo et al., 2025). These studies establish the broader measurement problem.

Our programming experiment asks a narrower accounting question. It holds the generated response and tests fixed, then changes only a predefined acceptance rule. For Gemma's SPEC-versus-NL comparison, this separates the part of the strict score gap that remains under tolerant scoring from the part created by differential wrapper acceptance. The contribution is therefore the controlled same-output accounting and the prospective reserved-task scaffold test, not the claim that output format can matter.

Work on code evaluation has shown that the test suite itself can change conclusions about correctness (Liu et al., 2023). Our comparison instead keeps the tests fixed and varies a narrowly defined acceptance rule. Both concerns fit a broader construct-validity question: whether an operational benchmark score supports the interpretation attached to it (Bean et al., 2025). Holistic evaluation makes scenarios, adaptation methods, and multiple metrics explicit (Liang et al., 2023). The present results provide two concrete reasons for that separation, one concerning conditional prediction and the other concerning interface compliance.

6 Implications and scope

6.1 Useful task information can coexist with strong general ability

The strongest predictive result is modest in size but demanding in its conditioning. The predictor already knows which of fourteen peers passed the item. Eighteen additional task characteristics nevertheless reduce held-out log loss, including when the target benchmark is absent from fitting. The result is compatible with shared general-performance structure and additional task-specific predictive information. It does not estimate a latent general-ability factor, and it does not require choosing between an entirely general account and an entirely task-specific one.

For evaluation researchers, the immediate use is to treat structured item descriptions as candidate inputs when modelling an existing response matrix, and to validate their increment at the intended transfer level. The result does not yet establish a routing policy, a post-training intervention, or a cost-saving deployment procedure. Such uses require a decision-level test with the information and costs available at deployment. In particular, a system that has not observed peer responses faces a different prediction problem.

6.2 Acceptance is part of the measured system

The programming result shows why score changes should be traced to observable failure types before being interpreted as capability changes. Improving interface compliance may be exactly what an application needs. It should not be described as improved problem solving without evidence at that endpoint. Conversely, mechanically accepting an otherwise usable function may reveal that part of a benchmark difference arose after generation rather than from the generated algorithm.

The distinction also guides practical interventions. When a large gap disappears under a fixed, content-preserving wrapper rule, a better prompt scaffold or an explicit adapter addresses a demonstrated interface problem. Additional reasoning or training has not been shown necessary for those recovered outputs. When a program still fails after valid extraction, the present wrapper intervention has not solved that failure. Keeping these cases separate prevents one remedy from being credited with solving a different problem.

6.3 What the two studies establish

The studies support an evaluation practice rather than a universal capability equation. Task descriptions are tested for the additional prediction they provide. Input and acceptance conditions are tested for the outcomes they change. Their evidential roles remain separate: Study 2 does not validate the task coordinates in Study 1, and Study 1 does not establish the programming mechanism. The synthesis is that both sources of variation belong in an account of observed performance, and neither should be inferred from a benchmark label alone.

The boundaries of the results are informative rather than incidental. The full text pipeline fails its primary comparison. The compact text comparison remains sensitive to regularisation. The semantic-success tests do not confirm a general specification benefit. These outcomes restrict the account to the parts the data support: a stable conditional annotation gain in one released battery, and a large, directly measurable acceptance effect in one controlled programming setting.

7 Limitations and reproducibility

Study 1 reuses ADeLe's selected items, rubric annotations, and binary outcome labels. We do not independently validate those labels or establish that each coordinate isolates a cognitive requirement. Only fifteen fixed target models and twenty benchmarks are represented. Benchmark holdout leaves possible content overlap and broader benchmark-ecosystem regularities intact. The saved-prediction bootstrap does not capture every source of fitting or model-population uncertainty.

Only one generic encoder and one downstream model family are compared in the transfer extension. In the original nested grid, all 300 full-embedding fits selected the strongest available penalty. A post hoc grid extension then added two stronger penalties. The new lower boundary was never selected, yet the full embedding remained worse than the response baseline. This removes the specific original-boundary explanation, but it does not rule out different encoders, separate penalties for feature blocks, or other estimators. Truncation was permitted in embedding requests and its frequency was not measured. Equal dimension does not equate representation quality or annotation cost. None of the predictive comparisons identifies a causal effect of task demands.

Study 2 uses two quantised local checkpoints selected through a development-task competence screen, one language, and one implementation interface. It estimates performance under a fixed greedy-decoding policy, not success averaged over independently sampled generations. Numerical reproducibility was not measured by repeated identical runs. Packaged test success does not establish correctness beyond those tests. The 72-task comparisons leave uncertainty about smaller semantic effects, and no equivalence margin was specified.

The supplementary material provides the full contrast tables, analysis-status record, configuration details, and numerical audit code. The accompanying audit data contain the complete 576-cell programming outcome grid and benchmark-level paired prediction summaries. The companion archive contains the original item-level predictions, confirmation metadata, response-only records, evaluation code, protocols, and the developmental audit with a portable reproduction entry point. No new candidate generations are required to verify the reported score accounting or exact paired tests. Supplementary Section S8 distinguishes verification from retained outcomes from a full rerun of inference and fitting.

8 Conclusion

Task information and evaluation conditions contribute different things to a model score. Published structured task characteristics add a small, stable predictive increment beyond a strong peer-response baseline on held-out benchmarks. In the Gemma programming comparison, the same saved responses produce a 25.0-point SPEC-versus-NL advantage under strict scoring but only 2.8 points when bare test-passing functions can be mechanically wrapped. Separating these effects changes the interpretation without requiring a new capability theory. The practical lesson is simple: establish what task information adds to prediction, then determine what presentation and acceptance rules add to the score before treating an observed difference as a difference in problem-solving capability.

References

Stephen Bates, Trevor Hastie, and Robert Tibshirani. Cross-validation: What does it estimate and how well does it do it? *Journal of the American Statistical Association*, 119(546):1434–1445, 2024. doi: 10.1080/01621459.2023.2197686. URL https://doi.org/10.1080/01621459.2023.2197686.

Andrew M. Bean, Ryan Othniel Kearns, Angelika Romanou, et al. Measuring what matters: Construct validity in large language model benchmarks. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://arxiv.org/abs/2511.04703. Datasets and Benchmarks Track.

Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Babu Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A. Huerta, and Hao Peng. Context length alone hurts LLM performance despite perfect retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 23281–23298. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.1264. URL https://aclanthology.org/2025.findings-emnlp.1264/.

Morten W. Fagerland, Stian Lydersen, and Petter Laake. The McNemar test for binary matched-pairs data: mid-p and asymptotic are better than exact conditional. *BMC Medical Research Methodology*, 13:91, 2013. doi: 10.1186/1471-2288-13-91. URL https://doi.org/10.1186/1471-2288-13-91.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437. URL https://doi.org/10.1198/016214506000001437.

Percy Liang, Rishi Bommasani, Tony Lee, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. URL https://arxiv.org/abs/2211.09110.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation, 2023. URL https://arxiv.org/abs/2305.01210. arXiv:2305.01210.

Do Xuan Long, Hai Nguyen Ngoc, Tiviatis Sim, Hieu Dao, Shafiq Joty, Kenji Kawaguchi, Nancy F. Chen, and Min-Yen Kan. LLMs are biased towards output formats! systematically evaluating and mitigating output format bias of LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 299–330. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.15. URL https://aclanthology.org/2025.naacl-long.15/.

Marcos Macedo, Yuan Tian, Filipe R. Cogo, and Bram Adams. Exploring the impact of the output format on the evaluation of large language models for code translation. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*, FORGE '24, pp. 57–68. Association for Computing Machinery, 2024. doi: 10.1145/3650105.3652301. URL https://doi.org/10.1145/3650105.3652301.

Marcos Macedo, Yuan Tian, Filipe R. Cogo, and Bram Adams. Output format biases in the evaluation of large language models for code translation, 2025. URL https://arxiv.org/abs/2403.17214v2. Expanded version, arXiv:2403.17214v2, 13 October 2025.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 34303–34326, 2024. URL https://proceedings.mlr.press/v235/maia-polo24a.html.

Aviya Maimon, Amir David Nisan Cohen, Gal Vishne, Shauli Ravfogel, and Reut Tsarfaty. From benchmarks to skills: Low-rank factors for LLM evaluation. *Transactions of the Association for Computational Linguistics*, 14:1660–1684, 2026. doi: 10.1162/tacl.a.745. URL https://aclanthology.org/2026.tacl-1.75/.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996. URL https://doi.org/10.1007/BF02295996.

Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of what art? a call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12:933–949, 2024. doi: 10.1162/tacl_a_00681. URL https://aclanthology.org/2024.tacl-1.52/.

Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic Embed: Training a reproducible long context text embedder. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=IPmzyQSiQE.

Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://arxiv.org/abs/2405.10938.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.11324.

Zichen Xie, Mrigank Pawagi, Yuxin Liu, Aaditi Rai, Lize Shao, John Berberian, Jr., Sicong Che, and Wenxi Wang. VeriContest: A competitive-programming benchmark for verifiable code generation, 2026. URL https://arxiv.org/abs/2605.08553v1. arXiv:2605.08553v1.

Lexin Zhou, Lorenzo Pacchiardi, Fernando Martínez-Plumed, et al. General scales unlock AI evaluation with explanatory and predictive power. *Nature*, 652:58–67, 2026. doi: 10.1038/s41586-026-10303-2. URL https://doi.org/10.1038/s41586-026-10303-2.